PR1(DFS BFS)

```
def dfs(graph, visited, node):
    if node not in visited:
        visited.append(node)
        for n in graph[node]:
            dfs(graph, visited, n)
    return visited
graph = {'A': ['B', 'C'],
         'B': ['D', 'E'],
         'C': ['F'],
         'D': [],
         'E': ['F'],
         'F': []}
visited = dfs(graph, [], 'A')
print(visited)
def bfs(graph, visited, queue):
    if queue:
        node = queue.pop(0)
        if node not in visited:
            visited.append(node)
            for n in graph[node]:
                queue.append(n)
        bfs(graph, visited, queue)
    return visited
graph = {'A': ['B', 'C'],
         'B': ['D', 'E'],
         'C': ['F'],
         'D': [],
         'E': ['F'],
         'F': []}
visited = bfs(graph, [], ['A'])
print(visited)
```

PR2 (A* Algorithm)

```
import copy
from heapq import heappush, heappop
n = 3
# bottom, left, top, right
row = [ 1, 0, -1, 0 ]
col = [ 0, -1, 0, 1 ]
class priorityQueue:
    def __init__(self):
        self.heap = []

    def push(self, k):
        heappush(self.heap, k)

    def pop(self):
        return heappop(self.heap)

    def empty(self):
        if not self.heap:
            return True
        else:
            return False
class node:
    def __init__(self, parent, mat, empty_tile_pos, cost, level):
        self.parent = parent
        self.mat = mat
        self.empty_tile_pos = empty_tile_pos
        self.cost = cost
        self.level = level
    def __lt__(self, nxt):
        return self.cost < nxt.cost

def calculateCost(mat, final) -> int:
    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and
                (mat[i][j] != final[i][j])):
                count += 1
    return count

def newNode(mat, empty_tile_pos, new_empty_tile_pos,level, parent, final) -> node:
```

```
        new_mat = copy.deepcopy(mat)
        x1 = empty_tile_pos[0]
        y1 = empty_tile_pos[1]
        x2 = new_empty_tile_pos[0]
        y2 = new_empty_tile_pos[1]
        new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]
        cost = calculateCost(new_mat, final)
        new_node = node(parent, new_mat, new_empty_tile_pos,cost, level)
        return new_node

def printMatrix(mat):
        for i in range(n):
                for j in range(n):
                        print("%d " % (mat[i][j]), end = " ")
                print()

# Function to check if (x, y) is a valid  matrix coordinate
def isSafe(x, y):
        return x >= 0 and x < n and y >= 0 and y < n

# Print path from root node to destination node
def printPath(root):
        if root == None:
                return

        printPath(root.parent)
        printMatrix(root.mat)
        print()

# the blank tile position in the initial state.
def solve(initial, empty_tile_pos, final):

        # Create a priority queue to store live nodes of search tree
        pq = priorityQueue()
        # Create the root node
        cost = calculateCost(initial, final)
        root = node(None, initial, empty_tile_pos, cost, 0)
        # Add root to list of live nodes
        pq.push(root)
        while not pq.empty():

                # Find a live node with least estimated cost and delete it from the list of live
nodes
```

```
        minimum = pq.pop()
        # If minimum is the answer node
        if minimum.cost == 0:

            # Print the path from root to destination;
            printPath(minimum)
            return

        # Generate all possible children
        for i in range(4):
            new_tile_pos = [
                minimum.empty_tile_pos[0] + row[i],
                minimum.empty_tile_pos[1] + col[i], ]
            if isSafe(new_tile_pos[0], new_tile_pos[1]):
                # Create a child node
                child = newNode(minimum.mat,
                                minimum.empty_tile_pos,
                                new_tile_pos,
                                minimum.level + 1,
                                minimum, final,)
                # Add child to list of live nodes
                pq.push(child)
# Driver Code
# Initial configuration Value 0 is used for empty space
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]
# Solvable Final configuration Value 0 is used for empty space
final = [ [ 1, 2, 3 ],
    [ 5, 8, 6 ],
    [ 0, 7, 4 ] ]
# Blank tile coordinates in initial configuration
empty_tile_pos = [ 1, 2 ]
# Function call to solve the puzzle
solve(initial, empty_tile_pos, final)e_pos, final)
```

PR3 (Greedy search algorithm for the Selection Sort)

```
def selectionSort(array, size):

    for ind in range(size):
        min_index = ind

        for j in range(ind + 1, size):
            # select the minimum element in every iteration
            if array[j] < array[min_index]:
                min_index = j
         # swapping the elements to sort the array
        (array[ind], array[min_index]) = (array[min_index], array[ind])

arr = [34,20,3,14]
size = len(arr)
selectionSort(arr, size)
print('The array after sorting in Ascending Order by selection sort is:')
print(arr)
```

PR4 n-queens

```
# Taking number of queens as input from user
print ("Enter the number of queens")
N = int(input())

# here we create a chessboard
# NxN matrix with all elements set to 0
board = [[0]*N for _ in range(N)]

def attack(i, j):
    #checking vertically and horizontally
    for k in range(0,N):
        if board[i][k]==1 or board[k][j]==1:
            return True
    #checking diagonally
    for k in range(0,N):
        for l in range(0,N):
            if (k+l==i+j) or (k-l==i-j):
                if board[k][l]==1:
                    return True
    return False

def N_queens(n):
    if n==0:
        return True
    for i in range(0,N):
        for j in range(0,N):
            if (not(attack(i,j))) and (board[i][j]!=1):
                board[i][j] = 1
                if N_queens(n-1)==True:
                    return True
                board[i][j] = 0

    return False

N_queens(N)
for i in board:
    print (i)
```

PR5

```
def banking_chatbot():
    print("Welcome to SmartBank! How can I assist you today?")
    while True:
        print("\nYou can ask me the following:")
        print("1. What's my account balance?")
        print("2. What loan options do you offer?")
        print("3. How can I contact customer support?")
        print("4. What are your bank's opening hours?")
        print("5. How do I reset my password?")
        print("6. Exit")
        # Get user's query
        question = input("Please enter your question: ").lower()
        if "account balance" in question:
            print("\nYour account balance is $2,500.00.")
        elif "loan options" in question:
            print("\nWe offer the following loan options:")
            print("1. Personal Loan")
            print("2. Home Loan")
            print("3. Car Loan")
            print("For more details, please visit our website or contact customer support.")
        elif "contact customer support" in question:
            print("\nYou can reach our customer support team at:")
            print("Email: support@smartbank.com")
            print("Phone: 1-800-555-1234")
        elif "opening hours" in question:
            print("\nOur bank's opening hours are:")
            print("Monday to Friday: 9:00 AM - 5:00 PM")
            print("Saturday: 9:00 AM - 1:00 PM")
            print("Closed on Sundays and public holidays.")
        elif "reset my password" in question:
            print("\nTo reset your password, please follow these steps:")
            print("1. Visit our password reset page on the website.")
            print("2. Enter your registered email address.")
            print("3. Check your inbox for a password reset link.")
            print("4. Follow the instructions to set a new password.")
        elif "exit" in question:
            print("Thank you for using SmartBank. Have a great day!")
            break
        else:
            print("Sorry, I didn't understand that. Please ask again.")
banking_chatbot()
```

CC GCP :
sudo apt update && sudo apt upgrade -y
python3 –version
sudo apt install -y python3.9 python3.9-dev python3.9-distutils
sudo apt install python3-pip
Add both versions to alternatives
sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.8 1
sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.9 2
sudo update-alternatives --config python3

sudo apt-get install curl

echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg]
https://packages.cloud.google.com/apt cloud-sdk main" | sudo tee -a
/etc/apt/sources.list.d/google-cloud-sdk.list

curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key --
keyring /usr/share/keyrings/cloud.google.gpg add –

sudo apt update
sudo apt install google-cloud-sdk-app-engine-python -y
gcloud init
gcloud components list
mkdir hello-world-app
cd hello-world-app
sudo apt install python3.9-venv
python3 -m venv venv
source venv/bin/activate

1. Create app.yaml This file configures the App Engine environment:
runtime: python39
entrypoint: gunicorn -b :$PORT main:app

2. Create main.py This is the Python code for the "Hello World" app using Flask:
```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
    return 'Hello, World!'
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

3.Create requirements.txt List the dependencies:
Flask==2.3.3
gunicorn==20.1.0

pip install -r requirements.txt
sudo apt-get install google-cloud-cli-app-engine-python google-cloud-cli-datastore-emulator
dev_appserver.py app.yaml
Open a browser and visit http://localhost:8080. You should see "Hello, World!".

Step 4: Deploy to Google App Engine
1. Deploy the app:
run on terminal
gcloud app deploy app.yaml

o Confirm the deployment when prompted.
2. Once deployed, get the URL:
run on terminal
gcloud app browse

AWS :-

df -h
lsblk
sudo file -s /dev/xvdf
sudo mkfs -t xfs /dev/xvdf
sudo mkdir -p /apps/volume/new-volume
sudo mount /dev/xvdf /apps/volume/new-volume
df -h